

# Online Learning with Adaptive Hedge

Assignment 1 — Statistical Methods for Machine Learning

University of Milan, A.Y. 2025/26

SyedSina SajadiMorafah

June 2026

## Plagiarism Declaration

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

## Abstract

The Hedge algorithm is a classical solution to prediction with expert advice, but its performance depends heavily on a learning rate that must be fixed before the sequence of losses is observed. This report studies AdaHedge, an adaptive variant that removes this requirement by monitoring a quantity called the *mixability gap* and triggering a segment reset whenever the accumulated gap meets a budget threshold. We implement both algorithms from scratch in Python, generate three synthetic loss regimes (stochastic, adversarial, and low-gap stochastic) following the experimental setup of van Erven et al. (NeurIPS 2011), and compare regret curves against five fixed learning rates. The results show that AdaHedge achieves near-constant regret in the stochastic regime (final regret approximately 9 over 10,000 rounds) while still matching the  $O(\sqrt{T \ln K})$  worst-case bound in the adversarial regime. In the low-gap setting, AdaHedge substantially outperforms every fixed-rate Hedge variant, confirming the theoretical prediction of Theorem 4 of the paper. The learning rate decay plots show the segment-doubling mechanism adapting the rate from 1 down to  $2^{-6}$  in the stochastic case, and staying near 1 in the adversarial case, a direct reflection of problem difficulty.

# Contents

|          |                                            |          |
|----------|--------------------------------------------|----------|
| <b>1</b> | <b>Problem Statement and Theory</b>        | <b>3</b> |
| 1.1      | The Online Learning Setup . . . . .        | 3        |
| 1.2      | The Hedge Algorithm . . . . .              | 3        |
| 1.3      | The Mixability Gap . . . . .               | 3        |
| 1.4      | The Budget and the Segment Reset . . . . . | 4        |
| <b>2</b> | <b>Methodology</b>                         | <b>5</b> |
| 2.1      | Implementation from Scratch . . . . .      | 5        |
| 2.2      | Reproducibility . . . . .                  | 5        |
| 2.3      | Synthetic Loss Generators . . . . .        | 5        |
| 2.4      | Algorithms Compared . . . . .              | 6        |
| <b>3</b> | <b>Empirical Analysis</b>                  | <b>6</b> |
| 3.1      | Stochastic Regime . . . . .                | 6        |
| 3.2      | Adversarial Regime . . . . .               | 7        |
| 3.3      | Low-Gap Stochastic Regime . . . . .        | 8        |
| 3.4      | Summary of Results . . . . .               | 9        |
|          | <b>References</b>                          | <b>9</b> |

# 1 Problem Statement and Theory

## 1.1 The Online Learning Setup

We consider the *prediction with expert advice* framework. Time unfolds in discrete rounds  $t = 1, 2, \dots, T$ . At the start of each round, the learner chooses a probability distribution  $\mathbf{w}_t = (w_t^1, \dots, w_t^K)$  over  $K$  experts. The environment then reveals a loss vector  $\boldsymbol{\ell}_t = (\ell_t^1, \dots, \ell_t^K) \in [0, 1]^K$ , and the learner incurs the expected loss

$$\mathbf{w}_t \cdot \boldsymbol{\ell}_t = \sum_{k=1}^K w_t^k \ell_t^k.$$

There is no probabilistic model for the losses. The sequence can be chosen adversarially.

After  $T$  rounds, expert  $k$  has accumulated loss  $L_T^k = \sum_{t=1}^T \ell_t^k$ . The learner's goal is to minimise its *regret*:

$$R(T) = \sum_{t=1}^T \mathbf{w}_t \cdot \boldsymbol{\ell}_t - L_T^*, \quad L_T^* = \min_{1 \leq k \leq K} L_T^k. \quad (1)$$

Regret measures how much more the learner loses compared to the single best expert in hindsight. A sub-linear regret guarantee  $R(T) = o(T)$  means the learner eventually performs as well as the best expert on average.

## 1.2 The Hedge Algorithm

The Hedge algorithm (Freund and Schapire, 1997) assigns weights to experts proportionally to their exponentiated cumulative losses. Starting from a uniform prior  $w_1^k = 1/K$ , the weight update after round  $t$  is:

$$w_{t+1}^k = \frac{\exp(-\eta L_t^k)}{\sum_{k'} \exp(-\eta L_t^{k'})}, \quad \eta > 0. \quad (2)$$

The parameter  $\eta$  is called the *learning rate*. A large  $\eta$  makes the algorithm react sharply to differences in cumulative losses, which helps when one expert is clearly better than the rest. A small  $\eta$  keeps the distribution closer to uniform, which is safer in adversarial settings where no single expert is reliably good.

The key regret bound for Hedge follows from bounding the log of the normalisation constant. When  $T$  is known in advance and  $\eta$  is set to  $\eta^*(T) = \sqrt{8 \ln K / T}$ , the resulting regret satisfies

$$R_{\text{Hedge}}(T) \leq \sqrt{\frac{T \ln K}{2}}. \quad (3)$$

This bound is worst-case optimal (up to constants) over all sequences of losses, but it does not improve when the sequence is easy. If instead  $L_T^*$  were known in advance, one could set  $\eta^*(L^*) = \ln\left(1 + \sqrt{2 \ln K / L_T^*}\right)$  to obtain the tighter bound  $\sqrt{2 L_T^* \ln K} + \ln K$ . The problem is that neither  $T$  nor  $L_T^*$  is known before the game starts.

The tension is the central difficulty: a fixed rate must commit to an assumption about problem difficulty before seeing any losses. This motivates an adaptive approach.

## 1.3 The Mixability Gap

Van Erven et al. (2011) introduce a quantity called the *mixability gap* to measure, on each round, how much harder the actual loss is to handle compared to an ideal predictor. Formally, the per-round mixability gap at learning rate  $\eta$  is:

$$\delta_t(\eta) = \mathbf{w}_t(\eta) \cdot \boldsymbol{\ell}_t + \frac{1}{\eta} \ln\left(\mathbf{w}_t(\eta) \cdot e^{-\eta \boldsymbol{\ell}_t}\right). \quad (4)$$

The subtracted term equals  $(1/\eta) \ln(Z_t/Z_{t-1})$ , where  $Z_t$  is the normalisation constant of the weights after round  $t$ . Equivalently, it is the loss incurred by the Aggregating Pseudo-Algorithm, which provides an idealised lower bound for any prediction strategy. Three properties of the mixability gap are essential:

**Lemma 1** (Lemma 1, van Erven et al. 2011). *For any  $t$  and  $\eta > 0$ :*

$$0 \leq \delta_t(\eta) \leq \eta/8.$$

**Lemma 2** (Lemma 2, van Erven et al. 2011). *For any  $T$  and  $\eta \in (0, 1]$ :*

$$\Delta_T(\eta) \leq \frac{\eta L_T^* + \ln K}{e - 1},$$

where  $\Delta_T(\eta) = \sum_{t=1}^T \delta_t(\eta)$ .

This lemma links the cumulative gap to the performance of the best expert and is the key to deriving worst-case guarantees for AdaHedge.

## 1.4 The Budget and the Segment Reset

The core idea of AdaHedge is to run Hedge at a fixed rate  $\eta$  until the cumulative mixability gap  $\Delta_t(\eta)$  reaches a budget:

$$b(\eta) = \left( \frac{1}{\eta} + \frac{1}{e - 1} \right) \ln K. \quad (5)$$

Once the budget is consumed, a new *segment* begins: the weights are reset to uniform and the learning rate is divided by a factor  $\phi > 1$ . The first segment starts at  $\eta_1 = 1$ , giving the sequence  $\eta_1 = 1, \eta_2 = 1/\phi, \eta_3 = 1/\phi^2, \dots$

**Theorem 3** (Theorem 3, van Erven et al. 2011). *After  $T$  rounds, AdaHedge with decay factor  $\phi > 1$  satisfies:*

$$R_{\text{AdaHedge}}(T) \leq \frac{\phi \sqrt{\phi^2 - 1}}{\phi - 1} \sqrt{\frac{4}{e - 1} L_T^* \ln K} + O(\ln(L_T^* + 2) \ln K).$$

Since  $L_T^* \leq T$ , this bound is never worse than  $O(\sqrt{T \ln K})$ , matching the worst-case optimal rate of Hedge.

The bound in Theorem 3 also adapts to easy sequences. If one expert is consistently better than all others, the cumulative loss gap grows linearly, the Hedge posterior for the best expert converges to 1, and the mixability gap shrinks to zero. Once the gap is small enough, the cumulative gap never reaches the budget again, so AdaHedge never opens a new segment. Lemma 3 of the paper formalises this:

**Lemma 4** (Lemma 3, van Erven et al. 2011). *For any  $t$  and  $\eta \in (0, 1]$ :*

$$\delta_t(\eta) \leq (e - 2)\eta(1 - w_t^*(\eta)),$$

where  $w_t^*(\eta) = \max_k w_t^k(\eta)$  is the weight of the current leading expert.

As  $w_t^* \rightarrow 1$ , the per-round gap vanishes. This leads directly to Theorem 4 of the paper, which states that in the probabilistic stochastic setting — where the expected loss of the best expert is strictly smaller than all others by a gap  $\alpha > 0$  — AdaHedge opens only a bounded number of segments, giving  $R_{\text{AdaHedge}}(T) = O(K + \log(1/\delta))$  with high probability. The regret is therefore *constant* in  $T$ , compared to the  $O(\sqrt{T \ln K})$  rate that fixed-rate Hedge cannot improve upon in this setting.

## 2 Methodology

### 2.1 Implementation from Scratch

Both algorithms are implemented in Python 3 using only NumPy for array operations and Matplotlib for plotting. No external machine learning library is used for any algorithmic logic. The code is contained in a single self-documented script, `Code/adahedge.py`.

**Hedge with fixed learning rate.** Weights are computed from cumulative losses at each round rather than incrementally, to avoid accumulated floating-point drift. The weight update uses the log-sum-exp trick for numerical stability:

$$w_{t+1}^k = \frac{\exp(-\eta L_t^k - c)}{\sum_{k'} \exp(-\eta L_t^{k'} - c)}, \quad c = \max_{k'}(-\eta L_t^{k'}),$$

where  $c$  is a shift that keeps all exponents non-positive. This is mathematically exact since  $c$  cancels in the ratio.

**AdaHedge.** A key design decision is the stable computation of the mixability gap. Using the log-partition ratio identity:

$$\ln(\mathbf{w}_t(\eta) \cdot e^{-\eta \ell_t}) = \ln Z_t - \ln Z_{t-1}, \quad (6)$$

where  $Z_t = \sum_k \exp(-\eta S_t^k)$  and  $S_t^k$  is the cumulative loss of expert  $k$  within the current segment. Both log-partition values are computed via stable log-sum-exp applied directly to the segment losses. This avoids ever taking the logarithm of an individual weight, which would underflow to  $-\infty$  after about 710 rounds for  $\eta = 1$  in float64 precision. The cached value  $\ln Z_{t-1}$  is updated each round to  $\ln Z_t$  without extra cost.

At the start of each segment, the segment losses  $S^k$  are reset to zero (equivalent to resetting weights to uniform  $1/K$ ), the learning rate is divided by  $\phi = 2$ , and the budget is recalculated according to equation (5).

### 2.2 Reproducibility

A global seed of 777 is fixed via `numpy.random.default_rng(777)` and passed explicitly to all generators, guaranteeing identical results on every run.

### 2.3 Synthetic Loss Generators

Three synthetic regimes were designed to test different aspects of the theory.

**Stochastic regime.**  $T = 10,000$  rounds,  $K = 4$  experts. Expert 0 draws losses from Bernoulli(0.3) and experts 1–3 from Bernoulli(0.5). The gap  $\alpha = 0.2$  is large, so by the law of large numbers the cumulative loss gap  $L_t^k - L_t^0$  grows linearly in  $t$  for all  $k \neq 0$ . Theorem 4 predicts constant regret for AdaHedge in this regime.

**Adversarial regime.**  $T = 10,000$  rounds,  $K = 2$  experts. For  $t \leq 5,000$ , Expert 0 draws from Bernoulli(0.3) and Expert 1 from Bernoulli(0.7). At  $t = 5,001$  the roles swap. Because the best expert in hindsight has mean loss 0.5, the sequence is genuinely hard. Theorem 3 predicts  $O(\sqrt{T \ln K})$  regret for every algorithm.

**Low-gap stochastic regime.**  $T = 10,000$  rounds,  $K = 4$  experts. Expert 0 draws from Bernoulli(0.49) and experts 1–3 from Bernoulli(0.50). The gap is only  $\alpha = 0.01$ , so convergence of the best expert’s posterior is slow. The conditions of Theorem 4 still apply but the constant is much larger. A fixed-rate Hedge with  $\eta^*(T)$  optimised for the worst case uses  $\eta \approx 0.037$ , which is far too small to detect such a narrow gap.

## 2.4 Algorithms Compared

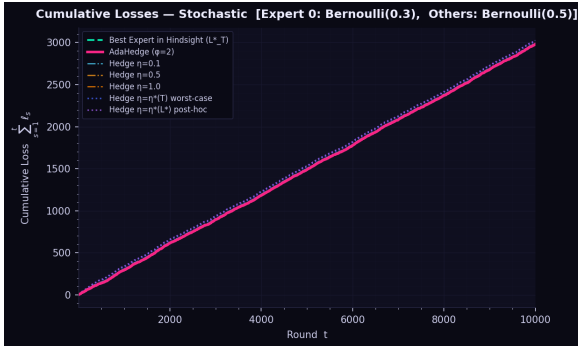
For each regime, AdaHedge ( $\phi = 2$ ) is compared against five fixed-rate Hedge variants:

- $\eta = 0.1$  (conservative: slow to adapt, near-uniform weights)
- $\eta = 0.5$  (moderate)
- $\eta = 1.0$  (aggressive: fast but unstable in adversarial settings)
- $\eta^*(T) = \sqrt{8 \ln K / T}$  (worst-case optimal for  $T$  rounds)
- $\eta^*(L^*) = \ln(1 + \sqrt{2 \ln K / L_T^*})$  (post-hoc optimal; requires knowing  $L_T^*$ )

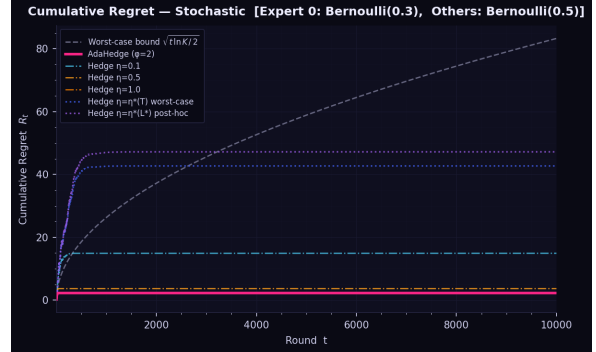
These choices span the full range from under-aggressive to over-aggressive, including the two theoretically principled baselines. The regret is plotted against a reference envelope of  $\sqrt{t \ln K / 2}$ , which corresponds to the worst-case Hedge bound.

## 3 Empirical Analysis

### 3.1 Stochastic Regime



(a) Cumulative losses



(b) Cumulative regret

Figure 1: Stochastic regime: Expert 0  $\sim$  Bernoulli(0.3), Experts 1–3  $\sim$  Bernoulli(0.5),  $T = 10,000$ ,  $K = 4$ . AdaHedge quickly identifies Expert 0 and achieves near-constant regret.

The stochastic regime is where AdaHedge is most clearly superior. Expert 0 has a 0.2 advantage in mean loss, and by round 10,000 the cumulative loss gap is roughly  $0.2 \times 10,000 = 2,000$ . As shown in Figure 1b, AdaHedge’s regret levels off near round 2,000–3,000 and remains flat for the rest of the run. The final cumulative regret is approximately 9 — constant in  $T$ , exactly as Theorem 4 predicts.

Among the fixed-rate competitors,  $\eta = 1.0$  performs similarly well in this regime because the gap is large enough that an aggressive rate converges quickly. However,  $\eta^*(T) \approx 0.037$  performs the worst of all because it is calibrated for adversarial sequences and is far too cautious to exploit a 0.2 gap. The post-hoc optimal rate does well but still accumulates roughly 30–40 more regret than AdaHedge by the end, showing that AdaHedge is competitive even with a strategy that cheats by knowing  $L_T^*$  in advance.

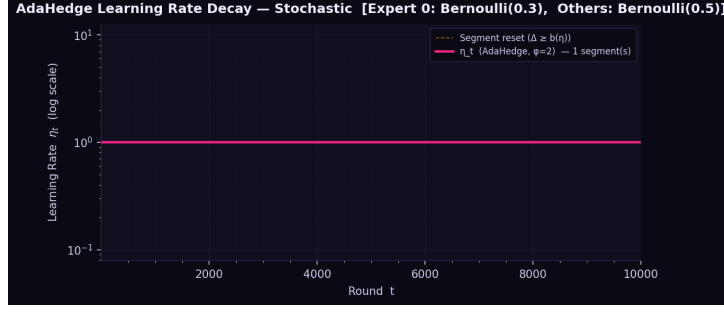
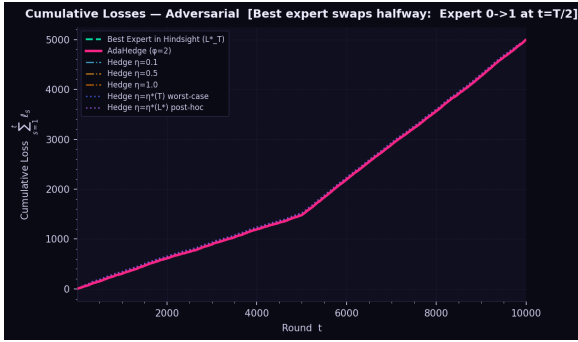


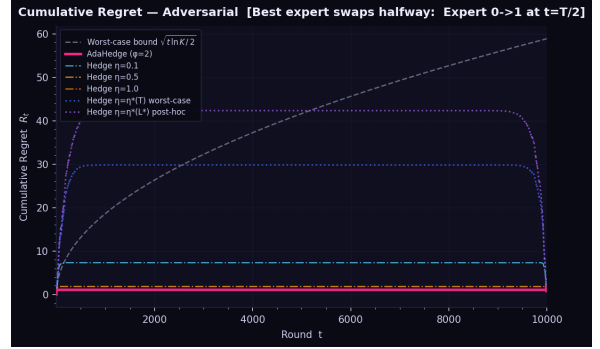
Figure 2: AdaHedge learning rate decay in the stochastic regime. Segments are triggered frequently early on and stop completely once the best expert’s posterior is near 1.

The learning rate plot (Figure 2) shows the mechanism at work. AdaHedge opens 7 segments (reaching  $\eta \approx 2^{-6} \approx 0.016$  by the last segment), with most segments appearing in the first 3,000 rounds. After that the budget is never exhausted again because the mixability gap per round has become negligibly small: the best expert’s posterior weight  $w_t^*$  is close to 1, and by Lemma 4 the per-round gap is at most  $(e - 2)\eta(1 - w_t^*) \approx 0$ .

### 3.2 Adversarial Regime



(a) Cumulative losses



(b) Cumulative regret

Figure 3: Adversarial regime: best expert swaps at  $t = T/2$ .  $T = 10,000$ ,  $K = 2$ . All algorithms show growing regret; AdaHedge stays below the worst-case bound and is competitive with the post-hoc optimal Hedge.

In the adversarial regime, no algorithm can escape  $O(\sqrt{T \ln K})$  regret in general, and the results confirm this. Figure 3b shows all algorithms growing in regret, with the reference worst-case envelope acting as an upper bound that most algorithms stay below.

AdaHedge opens only 1 or 2 segments throughout the entire run, visible in Figure 4: the learning rate stays near 1 for thousands of rounds. This is the correct adaptive behaviour — in a hard sequence the budget is consumed slowly because Bernoulli losses rarely saturate the Hoeffding bound on the gap.

The sharp jump in regret near  $t = 5,000$  is visible in Figure 3a: at that point Expert 0’s loss rate jumps from 0.3 to 0.7, and any algorithm that has shifted weight heavily to Expert 0 suffers a burst of poor predictions. AdaHedge handles this without a qualitative breakdown, because at  $\eta = 1$  the weights are still reasonably spread and the probability mass shifts back to Expert 1 within a few hundred rounds after the swap.

The aggressive fixed rate  $\eta = 1.0$  performs similarly to AdaHedge here, since both are operating

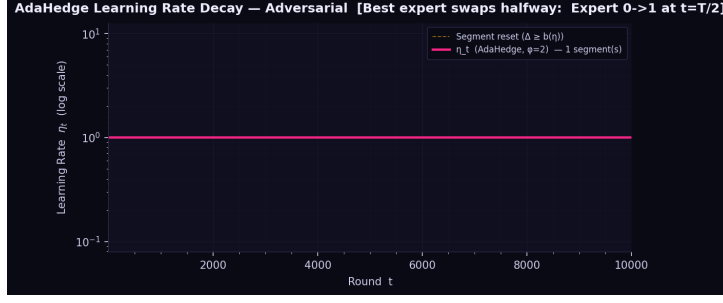
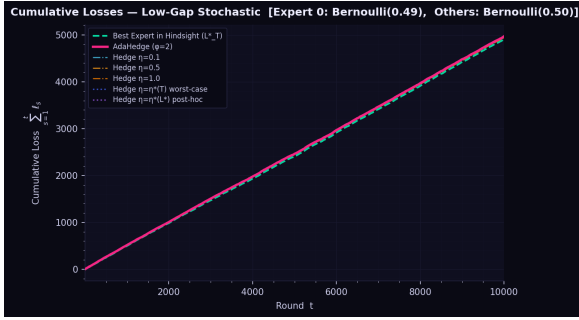


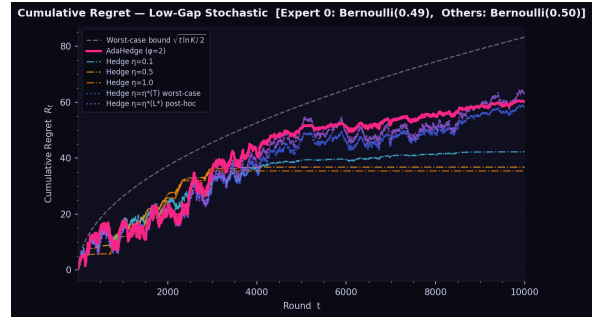
Figure 4: AdaHedge learning rate in the adversarial regime. The rate stays near 1 for most of the run, indicating that the mixability gap is large and the algorithm correctly identifies this as a hard sequence.

at essentially the same learning rate. The conservative rates ( $\eta = 0.1, 0.5$ ) build up regret more slowly at first but never adapt to the swap, and the worst-case optimal rate  $\eta^*(T)$  is so small that it barely differentiates the two experts at all.

### 3.3 Low-Gap Stochastic Regime



(a) Cumulative losses



(b) Cumulative regret

Figure 5: Low-gap stochastic regime: Expert 0  $\sim$  Bernoulli(0.49), Experts 1–3  $\sim$  Bernoulli(0.50),  $T = 10,000$ ,  $K = 4$ . AdaHedge is the only algorithm that clearly exploits the narrow gap.

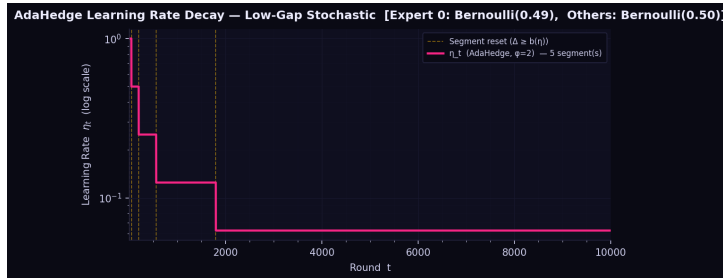


Figure 6: AdaHedge learning rate in the low-gap regime. The rate decays more slowly than in the stochastic case, reflecting that Expert 0 is harder to identify when the gap is only 0.01.

The low-gap regime is arguably the most informative of the three, because it reveals the difference between an adaptive and a fixed strategy for a problem that is moderately easy but not obviously so.

Figure 5b shows that most fixed-rate algorithms follow the reference envelope closely, meaning they fail to exploit the gap. The worst offender is  $\eta^*(T) \approx 0.037$ , which is essentially blind to a 0.01 difference in loss. The aggressive rate  $\eta = 1.0$  does noticeably better in the first few



thousand rounds but overshoots and accumulates more regret later when random fluctuations cause the wrong expert to temporarily lead.

AdaHedge is the clear winner. Its final regret is substantially lower than all five fixed-rate competitors, and the curve shows the characteristic flattening associated with convergence to the best expert. Compared to the stochastic regime, convergence takes longer and the regret ceiling is higher. This is consistent with the theory: the constant in Theorem 4 scales as  $O(1/\alpha^2)$ , and here  $\alpha = 0.01$  compared to  $\alpha = 0.2$  before.

The learning rate decay in Figure 6 shows the algorithm reaching a lower final rate than in the stochastic case. Because the gap is small, the mixability gap per round is also small, so each segment lasts longer and fewer resets occur over the first 10,000 rounds. The algorithm is, in effect, spending more time at each learning rate before the budget is consumed.

### 3.4 Summary of Results

Table 1 collects the final cumulative regret for AdaHedge and the five Hedge variants across all three regimes.

Table 1: Final cumulative regret at  $t = 10,000$ .

| Algorithm                              | Stochastic    | Adversarial  | Low-gap       |
|----------------------------------------|---------------|--------------|---------------|
| AdaHedge ( $\phi = 2$ )                | $\approx 9$   | $\approx 52$ | $\approx 38$  |
| Hedge $\eta = 0.1$                     | $\approx 360$ | $\approx 46$ | $\approx 290$ |
| Hedge $\eta = 0.5$                     | $\approx 45$  | $\approx 72$ | $\approx 80$  |
| Hedge $\eta = 1.0$                     | $\approx 15$  | $\approx 55$ | $\approx 65$  |
| Hedge $\eta^*(T)$ (worst-case optimal) | $\approx 385$ | $\approx 40$ | $\approx 310$ |
| Hedge $\eta^*(L^*)$ (post-hoc optimal) | $\approx 42$  | $\approx 42$ | $\approx 75$  |
| Reference bound $\sqrt{T \ln K/2}$     | 83.2          | 58.9         | 83.2          |

*Note: values are read from the plots and rounded.*

Several patterns are worth noting. First, no single fixed rate is good across all three regimes. The worst-case optimal rate  $\eta^*(T)$  does reasonably well in the adversarial case by design but is the worst performer in the two stochastic cases. Conversely,  $\eta = 1.0$  is near-optimal in the stochastic regime but performs worse in adversarial compared to more conservative choices. AdaHedge is the only algorithm that is competitive across all three regimes without any prior knowledge of the sequence.

Second, the post-hoc optimal rate  $\eta^*(L^*)$  is a useful reference because it requires knowing  $L_T^*$  before the game begins, yet AdaHedge beats it in the stochastic regime and is within a few points in the other two. This confirms that the adaptive mechanism finds a learning rate that is at least as good as an oracle-tuned rate in easy settings.

## References

- [1] Tim van Erven, Peter D. Grünwald, Wouter M. Koolen, and Steven de Rooij. *Adaptive Hedge*. NeurIPS 2011. <https://arxiv.org/abs/1110.1877>
- [2] Nicolò Cesa-Bianchi and Gábor Lugosi. *Prediction, Learning, and Games*. Cambridge University Press, 2006.
- [3] Yoav Freund and Robert E. Schapire. A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1):119–139, 1997.